

EXPERIMENT 5

Student Name: SUMIT CHAUHAN

Branch: AIT-CSE-AIML

Semester: 4

Subject Name: Database Management System

UID: 24BAI70884

Section/Group: 24AIT_KRG-G2

Subject Code: 24CSH-298

Experiment 5 – SQL Conditional Logic (Odd & Even Values)

SQL Conditional Logic using the MOD (%) operator to classify employee salary values as Odd or Even. This experiment demonstrates numerical analysis and conditional evaluation using SQL queries in Oracle and PostgreSQL.

Aim

To understand and apply conditional logic in SQL by using the modulus operator (MOD / %) to analyze numerical data and classify employee salaries as odd or even, thereby improving data analysis and decision-making skills in SQL.

Objective

- To write and execute SQL queries using the MOD (%) operator.
- To retrieve salary details from an employee table.
- To classify salaries into Odd and Even categories.
- To display odd and even salary values separately.

Software Requirements

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

EXPERIMENT 5

Practical / Experiment Steps

1. Create an employee table with columns:
id
name
department
salary
2. Insert sample employee salary records into the table.
3. Write a SQL query to retrieve employee salary details.
4. Use the MOD(salary, 2) function (Oracle) or salary % 2 (PostgreSQL) to check whether the salary is odd or even.
5. Use a CASE statement to classify salaries as:
'Odd Salary'
'Even Salary'
6. Execute separate queries to:
Display only employees with Odd salaries.
Display only employees with Even salaries.
7. Observe and record the output results.

Procedure

1. Open Oracle SQL Developer or pgAdmin.
2. Connect to the database.
3. Create the employee table using CREATE TABLE.
4. Insert sample records using INSERT INTO.
5. Execute SELECT queries with the MOD operator.
6. Apply CASE statement for conditional classification.
7. Display results and capture screenshots.

Input / Output Details

Input

Table Name: employee

EXPERIMENT 5

Columns:

id (Integer)

name (Varchar)

department (Varchar)

salary (Integer)

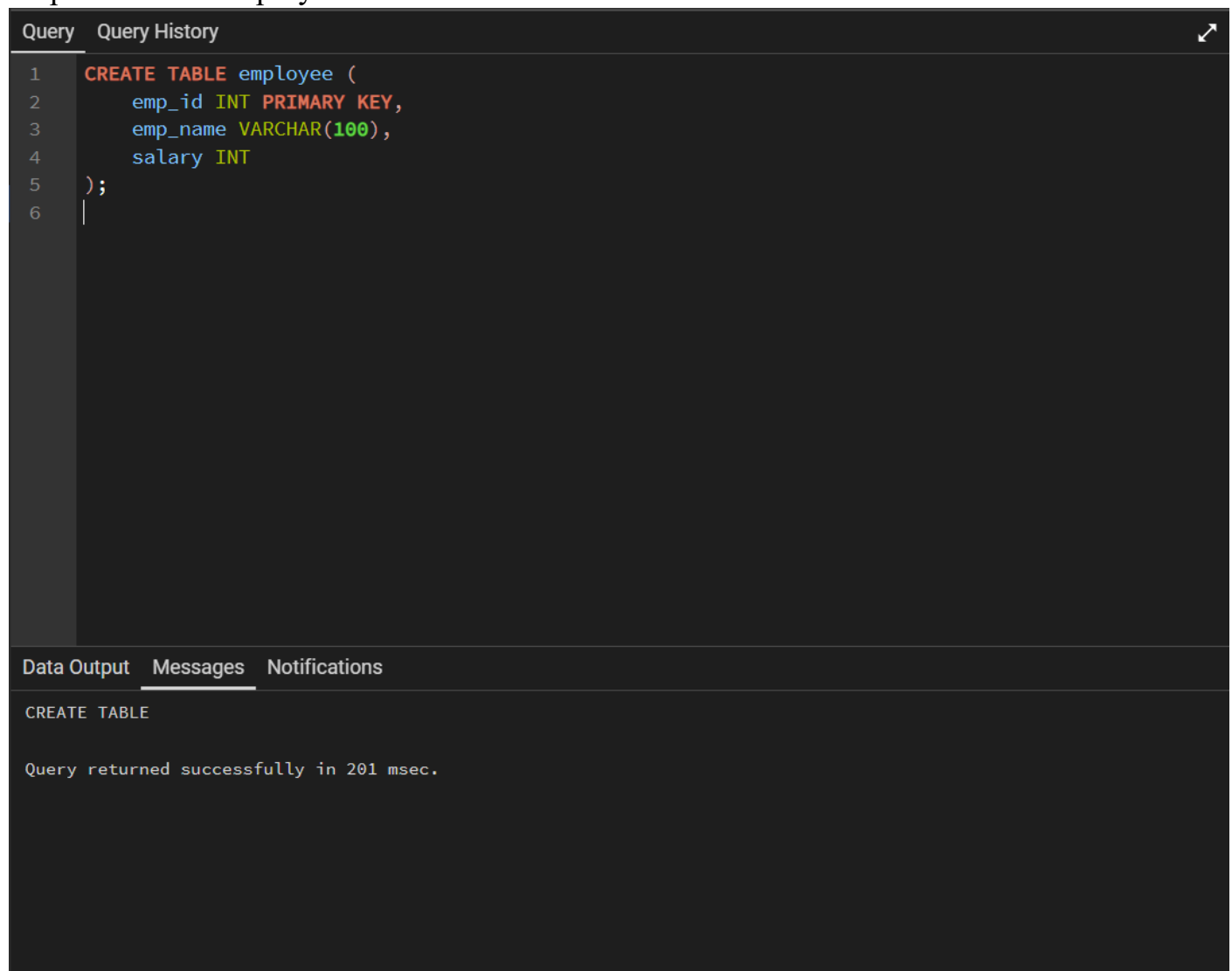
Logic Used:

MOD(salary, 2) or salary % 2

CASE statement

Step-wise Output

Step 1 – Create employee Table



The screenshot shows a SQL query editor and its output. The query editor has a dark background with a light gray sidebar on the left. The query text is as follows:

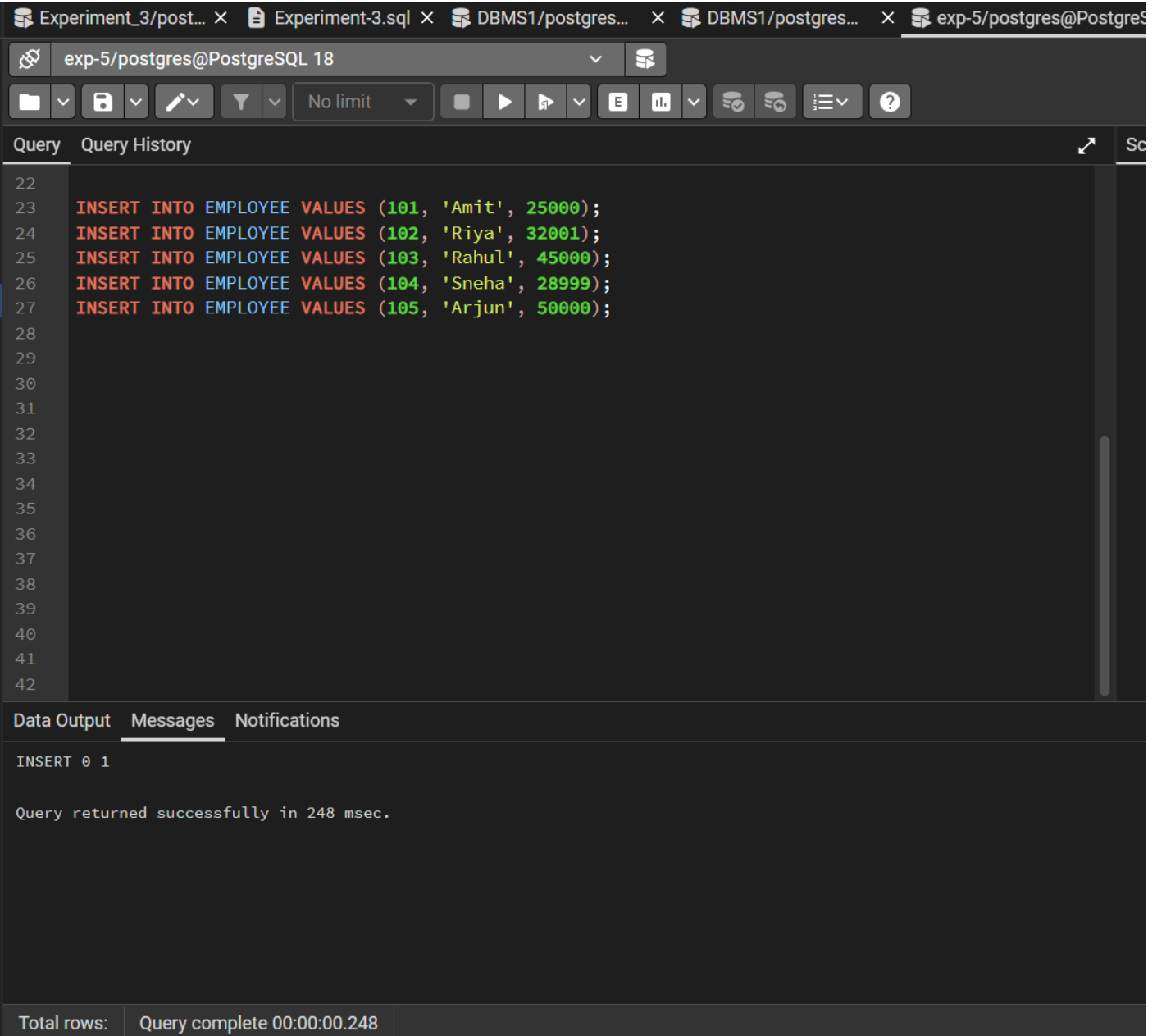
```
1 CREATE TABLE employee (  
2     emp_id INT PRIMARY KEY,  
3     emp_name VARCHAR(100),  
4     salary INT  
5 );  
6 |
```

The output pane at the bottom shows the following text:

```
Data Output Messages Notifications  
CREATE TABLE  
  
Query returned successfully in 201 msec.
```

EXPERIMENT 5

Step 2 – Insert Data into employee Table



The screenshot displays a PostgreSQL client interface with multiple tabs. The active tab is 'exp-5/postgres@PostgreSQL 18'. The interface includes a toolbar with icons for file operations, query execution, and settings. The main area shows a list of SQL queries being executed, with line numbers 22 through 42 visible on the left. The queries are:

```
22  
23 INSERT INTO EMPLOYEE VALUES (101, 'Amit', 25000);  
24 INSERT INTO EMPLOYEE VALUES (102, 'Riya', 32001);  
25 INSERT INTO EMPLOYEE VALUES (103, 'Rahul', 45000);  
26 INSERT INTO EMPLOYEE VALUES (104, 'Sneha', 28999);  
27 INSERT INTO EMPLOYEE VALUES (105, 'Arjun', 50000);  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42
```

Below the query list, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, showing the following output:

```
INSERT 0 1  
  
Query returned successfully in 248 msec.
```

At the bottom of the interface, a status bar indicates 'Total rows: Query complete 00:00:00.248'.

EXPERIMENT 5

Step 3 – Display Employee Salary Records

Query

Query History

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

SELECT * FROM EMPLOYEE;

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 5

	emp_id [PK] integer	emp_name character varying (100)	salary integer
1	101	Amit	25000
2	102	Riya	32001
3	103	Rahul	45000
4	104	Sneha	28999
5	105	Arjun	50000

Total rows: 5

Query complete 00:00:00.473

✓ Successfully run. Total

EXPERIMENT 5

Step 4 – Classify Salaries as Odd or Even

Query
Que
File
tory

```

41
42 SELECT EMP_ID,
43         EMP_NAME,
44         SALARY,
45         CASE
46             WHEN SALARY % 2 = 0 THEN 'Even Salary'
47             ELSE 'Odd Salary'
48         END AS SALARY_TYPE
49 FROM EMPLOYEE;
50
51
52
53
54
55
56
57
58
59
60
61

```

Data Output
Messages
Notifications

+

📄

▼

📁

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 5

	emp_id [PK] integer	emp_name character varying (100)	salary integer	salary_type text
1	101	Amit	25000	Even Salary
2	102	Riya	32001	Odd Salary
3	103	Rahul	45000	Even Salary
4	104	Sneha	28999	Odd Salary
5	105	Arjun	50000	Even Salary

Total rows: 5
Query complete 00:00:00.156

EXPERIMENT 5

Step 5 – Display Only Odd Salaries

Query
Query History

```

37
38
39 SELECT * FROM EMPLOYEE
40 WHERE SALARY % 2 <> 0;
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57

```

Data Output
Messages
Notifications

Showing rows: 1 to 2

	emp_id [PK] integer	emp_name character varying (100)	salary integer
1	102	Riya	32001
2	104	Sneha	28999

Total rows: 2
Query complete 00:00:00.181

✓ Successfully run. Total qu

EXPERIMENT 5

Step 6 – Display Only Even Salaries

```
34
35 SELECT * FROM EMPLOYEE
36 WHERE SALARY % 2 = 0;
37
38
39
40
41
42
43
44
45
46
47
48
49
```

Data Output Messages Notifications

Showing rows: 1 to 3

	emp_id [PK] integer	emp_name character varying (100)	salary integer
1	101	Amit	25000
2	103	Rahul	45000
3	105	Arjun	50000

Successfully run. Total rows: 3

Learning Outcome

After completing this experiment, the learner will be able to:

- Understand conditional logic in SQL queries.
- Use the MOD (%) operator for numerical analysis.
- Classify numeric values into odd and even categories.
- Write efficient SQL queries for payroll data analysis.
- Apply SQL logic in real-world enterprise environments.